- `increment_counter`
- `toggle`
- `touch`
- `update_column`
- `update_columns`
- `update_all`
- `update_counters`

These methods should be used with caution, however, because important business rules and application logic may be kept in callbacks. Bypassing them without understanding the potential implications may lead to invalid data.

6 Halting Execution

As you start registering new callbacks for your models, they will be queued for execution. This queue will include all your model's validations, the registered callbacks, and the database operation to be executed.

The whole callback chain is wrapped in a transaction. If any *before* callback method returns exactly `false` or raises an exception, the execution chain gets halted and a ROLLBACK is issued; *after*callbacks can only accomplish that by raising an exception.
Any exception that is not `ActiveRecord::Rollback` will be re-raised by Rails after the callback chain is halted. Raising an exception other than `ActiveRecord::Rollback` may break code that does not expect methods like `save` and `update_attributes` (which normally try to return `true` or `false`) to raise an exception.

7 Relational Callbacks

Callbacks work through model relationships, and can even be defined by them. Suppose an example where a user has many articles. A user's articles should be destroyed if the user is destroyed. Let's add an `after_destroy` callback to the `User` model by way of its relationship to the `Article` model:

```
class User < ActiveRecord::Base
  has_many :articles, dependent: :destroy
end

class Article < ActiveRecord::Base
  after_destroy :log_destroy_action

  def log_destroy_action
    puts 'Article destroyed'
  end
end

>> user = User.first
=> #<User id: 1>
>> user.articles.create!
=> #<Article id: 1, user_id: 1>
>> user.destroy
Article destroyed
=> #<User id: 1>
```

8 Conditional Callbacks

As with validations, we can also make the calling of a callback method conditional on the satisfaction of a given predicate. We can do this using the `:if` and `:unless` options, which can take a symbol, a string, a `Proc` or an `Array`. You may use the `:if` option when you want to specify under which conditions the